

Strategic Architecture for the AI-Powered HR Onboarding Document Assistant: A Comprehensive Technical and Operational Report

1. Executive Strategy: The Automation of Workforce Readiness

1.1 The Operational Crisis in Modern Human Resources

The contemporary enterprise landscape is characterized by a relentless demand for speed and agility, yet the fundamental processes governing the entry of talent into the organization—onboarding—remain paradoxically archaic. The onboarding phase is the critical "last mile" of recruitment, a fragile period where the psychological contract between employer and employee is solidified. However, operational reality often contradicts strategic intent. Human Resources (HR) departments, tasked with being strategic architects of culture, are frequently reduced to administrative gatekeepers, bogged down by the sheer volume of unstructured documentation required to bring a new hire into the fold.

Research indicates a startling inefficiency at the heart of this process: HR teams currently invest between 30 to 60 minutes per employee solely in the manual review of onboarding documents.¹ This is not merely a time sink; it is a structural bottleneck that introduces latency into workforce readiness. In high-volume hiring environments—such as retail, logistics, or rapid-growth technology sectors—this latency compounds, creating a "readiness gap" where employees are hired but legally or operationally unable to work. The manual extraction of key contract terms such as start dates, probation periods, and notice periods is prone to human error, leading to downstream data integrity issues in Human Resource Information Systems

(HRIS) like SAP SuccessFactors. Furthermore, the risk of non-compliance is elevated; missing signatures on Non-Disclosure Agreements (NDAs) or data protection acknowledgments can expose the organization to significant legal and regulatory liability.¹

The proposed solution, an **AI-Powered HR Onboarding Document Assistant (MVP)**, addresses these systemic failures not through incremental improvement, but through a paradigm shift powered by Agentic AI. By leveraging **IBM Watsonx Orchestrate** as the cognitive core, the solution aims to compress the document review cycle from 45 minutes to under one minute, while simultaneously increasing extraction accuracy to over 80% and compliance flagging to over 90%.¹ This report details the architectural, technical, and strategic roadmap for realizing this MVP, transforming onboarding from a bureaucratic hurdle into a seamless, automated, and data-driven competitive advantage.

1.2 The Shift to Agentic AI

The distinction between "Generative AI" and "Agentic AI" is the pivot point of this architecture. Standard generative models (LLMs) are passive; they wait for a prompt and generate text. Agentic systems, specifically those built on **Watsonx Orchestrate**, are active. They possess the capacity to perceive their environment, reason about data, and execute specific goals through tool use.¹

In the context of HR onboarding, a passive chatbot is insufficient. An onboarding assistant must do more than answer questions; it must *manage* the process. It needs to act as a specialized "Document Reader" to ingest PDF contracts, a "Compliance Auditor" to verify legal terms against company policy, and an "Action Orchestrator" to update backend systems.¹ This necessitates a move beyond simple text generation to a structured orchestration of specialized "skills"—modular units of logic that can interact with external systems like SAP SuccessFactors.

The architecture proposed herein utilizes **Watsonx Orchestrate** as the central nervous system, managing a constellation of specialized agents. This is supported by a robust hybrid technical stack featuring a **React/JSX** frontend for intuitive user interaction and a **Node.js Express** backend for secure API orchestration and state management. This combination ensures that while the AI provides the reasoning, the application maintains the robust, stateful, and secure environment required for enterprise software.

1.3 Defining Real-Time Value

"Real-time value" in HR is often an abstract concept, but this MVP concretizes it through immediate risk detection and readiness scoring. The system is designed to deliver a **Structured Onboarding Summary** seconds after document upload.¹ This summary acts as a live dashboard, dynamically updating as the AI processes the document stream.

Value Dimension	Current State (Manual)	Future State (AI-Powered MVP)
Latency	30-60 minutes per file	< 1 minute processing time
Accuracy	Variable, fatigue-dependent	Consistent, rule-based (>80%)
Compliance	Reactive, post-audit detection	Proactive, pre-submission flagging
Data Entry	Manual keying into HRIS	Automated extraction & validation
Experience	Administrative burden	Strategic "Exception Management"

By shifting the HR Business Partner's role from data extraction to decision validation, the organization unlocks significant operational capacity, allowing HR professionals to focus on the human elements of integration and culture building.

2. Architectural Blueprint: The Hybrid Cognitive Stack

To deliver the capabilities outlined above, a robust technical architecture is required. The solution must balance the stateless nature of modern AI services with the stateful requirements of a multi-turn onboarding workflow.

2.1 High-Level Architecture Overview

The architecture is stratified into four distinct layers, each serving a specific function in the data processing pipeline.

1. **The Presentation Layer (React + JSX):** This layer is responsible for the user experience. It provides the drag-and-drop interface for document uploads and the rich-text editors for drafting communications. It utilizes React's virtual DOM to efficiently render the complex, nested JSON structures returned by the AI, presenting them as a clean, interactive dashboard.
2. **The Orchestration Layer (Node.js + Express):** This is the "Backend for Frontend" (BFF). It serves as the secure gateway, handling authentication with IBM Cloud, managing API secrets, and crucially, maintaining the session state of the conversation.¹
3. **The Cognitive Layer (Watsonx Orchestrate & Watsonx.ai):** This is the intelligence engine. It hosts the foundational models (e.g., meta-llama/llama-3-2-11b-vision-instruct for vision, ibm/granite-13b-chat-v2 for text) and the orchestration logic that routes tasks to specific agents.¹
4. **The System of Record Layer (Enterprise Data):** This comprises the external systems, primarily **SAP SuccessFactors**, which are accessed via the pre-built Watsonx agents.¹

2.2 The Statelessness Challenge and the Node.js Solution

One of the most critical technical insights derived from the research material is that **IBM Watsonx.ai is stateless**. The documentation explicitly states: "IBM watsonx.ai is stateless - there is no automatic session or conversation history storage between API calls".¹ This presents a significant challenge for an onboarding assistant, which is inherently conversational and iterative. If an HR user uploads a contract and then asks, "What is the start date?", a stateless model will not remember the document it just processed.

To overcome this, the **Node.js Express** backend must implement a robust **Client-Side History Management** pattern.

- **Session Initialization:** When a user accesses the React application, the backend initializes a unique sessionId.
- **Context Storage:** The backend utilizes an in-memory data store (or a persistent store like Redis for production resilience) to hold the "Context Window" for that session. This context includes the extracted text from the uploaded documents and the transcript of the chat history.
- **Context Injection:** For every subsequent API call to Watsonx, the Node.js middleware retrieves the relevant context from the store and prepends it to the user's prompt. This allows the AI to "remember" the document contents without the platform itself storing

the data, thereby also enhancing data privacy.

2.3 Security and Credential Management

The architecture must rigorously adhere to security best practices, particularly regarding credential management. The research warns against the exposure of IBM Cloud credentials in public repositories or client-side code.¹

- **Server-Side Secrets:** All sensitive credentials—specifically the `WATSONX_API_KEY` and `WATSONX_PROJECT_ID`—are stored exclusively in the Node.js environment variables. They are never embedded in the React code or exposed to the browser.
 - **Token Exchange:** The Node.js server handles the exchange of the long-lived API Key for a temporary IAM Bearer Token. This token is valid for one hour.¹ The backend implements logic to check the expiration timestamp and refresh the token automatically, ensuring uninterrupted service without exposing the root key.
 - **Proxying:** The React frontend does not call Watsonx directly. Instead, it calls custom endpoints on the Node.js server (e.g., `/api/analyze`, `/api/chat`), which then proxy the request to IBM Cloud. This adds a layer of abstraction that allows for request validation, rate limiting, and data sanitization before the query ever reaches the AI.
-

3. The Cognitive Engine: Watsonx Orchestrate and Agentic Design

The core differentiator of this solution is its **Agentic** nature. Unlike a monolithic script, the system is composed of discrete, specialized agents coordinated by **Watsonx Orchestrate**. This allows for modular development, easier maintenance, and the ability to swap out specific capabilities without redesigning the entire system.

3.1 Agent Structure and Responsibilities

The research outlines a clear division of labor among four primary agent types within the MVP scope.¹ Understanding the specific role and "Style" of each agent is crucial for effective

implementation.

3.1.1 The Document Reader Agent

This agent is the "perceptual" unit of the system. Its primary responsibility is the ingestion of unstructured data.

- **Technology:** It leverages **Multimodal Models** such as meta-llama/llama-3-2-11b-vision-instruct or pixtral-12b.¹ These models can process visual inputs (PDFs, images) directly via Base64 encoding, allowing them to "see" the layout of a document.
- **Function:** It extracts key terms defined in the schema: Role, Start Date, Salary, Probation Period, and Notice Period.
- **Output:** It produces a structured JSON object, stripping away the legalese and retaining only the data points required for the HRIS.

3.1.2 The Compliance Review Agent

This agent acts as the "reasoning" unit. It does not just read; it evaluates.

- **Logic:** It compares the extracted JSON data against a set of compliance rules. These rules can be hardcoded for the MVP (e.g., "Probation must not exceed 6 months") or retrieved from a "Knowledge Base" of policy documents.¹
- **Flagging:** It is responsible for identifying "Missing" or "Unclear" elements. If a document requires a signature and the Vision model does not detect one, this agent flags it as a critical compliance gap.

3.1.3 The Action Suggestion Agent

This agent is the "planner." It bridges the gap between insight and execution.

- **Orchestration:** It maps the validated data to specific downstream tasks. For instance, if a "New Hire" is identified, it knows that the sequence of actions involves "Create User ID," "Assign Email," and "Provision Laptop."
- **Integration:** It utilizes the specific capabilities of the **SAP SuccessFactors Agents**

(detailed in Section 4) to recommend precise, system-level transactions.¹

3.1.4 The Communication Agent

This agent is the "interface." It handles the human-centric aspects of the workflow.

- **Tone and Style:** The research emphasizes a "friendly professional tone".¹ This agent is configured with a specific system prompt (instructions) that dictates its persona.
- **Drafting:** It generates the "Welcome Message Draft." By using parameters like temperature (set to a moderate level like 0.7), it ensures the output is warm and engaging rather than robotic.¹

3.2 Building Agents with Watsonx Orchestrate

The process of constructing these agents within the Watsonx platform involves defining several key attributes, as detailed in the hackathon guide.¹

- **Profile:** Each agent requires a clear description. For the Compliance Agent, the profile might be: "An expert in HR policy and employment law responsible for validating contract terms." This description helps the Orchestrator route queries correctly.
- **Style:** The agent's interaction style must be defined. For the Document Reader, the style is "factual and concise." For the Communication Agent, the style is "empathetic and professional."
- **Knowledge:** Agents can be grounded in specific documents. The Compliance Agent can be augmented with uploaded PDF policy documents, allowing it to answer questions like "What is the standard notice period for Germany?" by referencing the uploaded file rather than hallucinating.
- **Tools:** Agents are equipped with tools. The Action Agent's tools are the API connectors to SAP SuccessFactors.

3.3 The "Agentic" Workflow

The workflow is not linear but dynamic.

1. The **Document Reader** ingests a file.

2. It passes the data to the **Compliance Reviewer**.
3. The Reviewer finds a discrepancy (e.g., salary is below the minimum for the pay grade).
4. Instead of proceeding, the Reviewer loops back to the user via the **Communication Agent**: "The salary listed is below band. Should I flag this for review?"
5. Once resolved, the workflow proceeds to the **Action Suggestion Agent**.

This dynamic interplay, enabled by the Orchestrate platform's reasoning capabilities (ReAct logic), allows the system to handle exceptions gracefully, mimicking a human colleague rather than a rigid script.¹

4. Deep Dive: The SAP SuccessFactors Agent Ecosystem

To deliver the "Real-Time Value" promised in the query, the MVP must do more than read documents; it must prepare the data for the enterprise ecosystem. The research material provides a rich catalog of **SAP SuccessFactors Agents**.¹ A deep analysis of these agents reveals how they can be integrated into the onboarding workflow to create a comprehensive "Readiness" engine.

4.1 Core HR Data Management Agents

These agents form the backbone of the new hire's digital identity.

- **Employee Personal Details Agent:**
 - *Role:* "Rapidly retrieve and update personal details, oversee emergency contact information."
 - *Workflow Integration:* When the Document Reader parses the CV or Personal Data Form, it extracts fields like "Marital Status" or "Emergency Contact Name." The Action Agent maps these fields to the schema required by this SAP agent. The "value" delivered is the pre-validation of this data—ensuring the emergency contact has a valid phone number format before the HRBP even attempts to enter it into SAP.
- **Employee Address Agent:**
 - *Role:* "Distinguish between various address types, and seamlessly update address records."
 - *Workflow Integration:* New hires often have temporary and permanent addresses. The MVP can parse both from the contract. By utilizing the logic of this agent, the

system can prompt the user: "The contract lists a relocation address. Should I set this as the 'Mailing Address' effective from the Start Date?"

- **Employee Email & Phone Agents:**

- *Role:* "Identify and categorize email types... Distinguish between phone types."
- *Workflow Integration:* The MVP ensures that the personal email extracted from the application is categorized as "Personal" and queues a task to generate a "Business" email using the standard corporate format (e.g., firstname.lastname@company.com), which is then validated for uniqueness.

4.2 Organizational Structure and Placement Agents

Onboarding is fundamentally about placing an individual into a hierarchy.

- **Organization Structure Agent:**

- *Role:* "Gain a clear view of reporting structures... identify employee direct reports."
- *Workflow Integration:* The contract lists a "Reporting Manager." The MVP uses this agent to verify that the named manager acts as a valid "Supervisor" in SAP. If the manager has left or transferred, the system flags a "Reporting Line Risk," preventing a common onboarding error where new hires are assigned to "ghost" managers.

- **Internal Mobility Agent:**

- *Role:* "Retrieve business unit, pay grade, division, department, and location information."
- *Workflow Integration:* This is crucial for internal transfers. If the onboarding document is for an existing employee moving roles, this agent allows the MVP to pull their *current* data. The system can then highlight *only the changes* (e.g., "Department changing from Sales to Marketing"), reducing the review burden significantly.

4.3 Compensation and Compliance Agents

These agents handle the most sensitive and legally critical aspects of onboarding.

- **Compensation Agent:**

- *Role:* "Check current compensation details, and explore available benefits plan."
- *Workflow Integration:* The Document Reader extracts "Base Salary: \$100,000." The system references the Compensation Agent to validate if this figure falls within the approved "Pay Grade" for the specific "Job Code" associated with the role. This automated audit ensures pay equity compliance from Day One.

- **Employee Visa Agent:**
 - *Role:* "Provide instant access to... critical work authorization details."
 - *Workflow Integration:* If the Document Reader detects a non-domestic passport or address, the system automatically triggers a check via the Visa Agent. It asks: "Does this candidate have a valid work permit on file?" If not, it adds "Collect Visa Document" to the "Missing Compliance" list.¹

4.4 Talent and Recruitment Agents

These agents bridge the gap between "Candidate" and "Employee."

- **Talent Acquisition Manager Agent:**
 - *Role:* "Orchestrating job requisition creation... candidate tracking."
 - *Workflow Integration:* The MVP ensures the "Job Requisition ID" on the contract matches the one in the system. It closes the loop by suggesting the action: "Mark Candidate as Hired on Requisition #12345."
- **Candidate Management Agent:**
 - *Role:* "Tag top applicants... track candidates."
 - *Workflow Integration:* The system can verify if the new hire was tagged as a "High Potential" or "Referral" during the interview process, prompting the HRBP to add specific onboarding steps (e.g., "Schedule Executive Welcome Lunch") tailored to VIP candidates.

4.5 Cross-Functional and Ancillary Agents

The research also lists agents that, while not strictly HR, are vital for holistic onboarding.

- **World Data Agent:**
 - *Role:* "Search for cities, states, and provinces by country."
 - *Workflow Integration:* This acts as a validation layer for all geographic data extracted from the documents. It prevents typos in city names or invalid state codes from polluting the master data.
- **Sales Engagement & Product Enablement Agents:**
 - *Role:* Manage access to Salesloft, HubSpot, Seismic.
 - *Workflow Integration:* If the "Role" is extracted as "Sales Executive," the Action Agent infers that access to these specific tools is required. It adds "Provision Salesloft Account" to the "Access & Equipment Required" section of the summary.¹ This moves

the MVP from "HR Tool" to "Enterprise Onboarding Engine."

5. Technical Mechanics: API Orchestration and State Management

The realization of this functionality relies on the precise implementation of the **IBM Watsonx.ai API**. A thorough understanding of the API's parameters and capabilities is essential for tuning the system's performance.

5.1 API Configuration and Parameters

The Node.js backend interacts with the API using a specific set of parameters that control the behavior of the model.¹

- **Model Selection (model_id):**
 - For extraction tasks, the architecture utilizes the **Granite** series (e.g., `ibm/granite-13b-chat-v2`) or **Llama** series (`meta-llama/llama-3-70b-instruct`). These models are optimized for instruction following.
 - For vision tasks, `meta-llama/llama-3-2-11b-vision-instruct` is the requisite choice for processing PDF content.
- **Generation Control:**
 - **temperature:** This parameter controls randomness. For the **Extraction Agent**, the temperature is set to 0 or 0.1.¹ This "Greedy Decoding" strategy forces the model to select the most probable token, which is critical for factual extraction where creativity is a liability (hallucination risk).
 - **repetition_penalty:** Set to 1.1 or 1.2. Contracts often contain repetitive legal phrasing. This penalty prevents the model from getting stuck in a loop of generating identical text.
 - **max_new_tokens:** Set to a limit appropriate for the task (e.g., 1024 for summaries, 200 for JSON fields) to manage latency and cost.
- **Response Control:**
 - **stop_sequences:** The backend sends sequences like `["}]"` or ``` to force the model to terminate generation cleanly once the JSON object is complete.¹

5.2 Implementing Retrieval-Augmented Generation (RAG)

To support the **Compliance Review Agent**, the architecture incorporates a RAG pattern. The system must verify extracted terms against complex internal policies that are too large to fit in the context window.

- **Vector Database Integration:** The research highlights support for vector databases like **Milvus** and **Elasticsearch**.¹
- **Workflow:**
 1. Company policy documents (e.g., "Global Leave Policy.pdf") are ingested, chunked, and embedded into vectors using an embedding model (e.g., `ibm/slate-30m-english-rtrvr`).
 2. These vectors are stored in **Milvus**.
 3. When the Compliance Agent checks a "Maternity Leave" clause, it queries Milvus for the relevant policy chunks.
 4. The retrieved text is injected into the prompt context, allowing the model to validate the contract against the specific, authoritative rules of the organization.

5.3 The Node.js State Machine

As established, handling state is the responsibility of the Node.js layer. This implementation details the "Sliding Window" mechanism required for long conversations.¹

- **Token Management:** The granite-13b model has a finite context window (e.g., 4096 or 8192 tokens). A long contract combined with a long chat history will quickly exceed this.
- **Sliding Window Logic:** The Node.js middleware monitors the token count of the `conversation_history` array.
 - If `current_tokens > max_limit`, the system triggers a summarization routine.
 - It calls a smaller, faster model to "Summarize the conversation so far, retaining key facts about the candidate."
 - It replaces the oldest messages with this summary, freeing up context space for new interactions while retaining the essential "memory" of the session.

5.4 SDK and Programmatic Access

The backend utilizes the **Python SDK** (`ibm-watsonx-ai`) or the Node.js equivalent for robust

interaction.

- **Installation:** `pip install ibm-watsonx-ai` or `npm install @ibm-cloud/watsonx-ai`.
 - **Environment Variables:** The code initializes the APIClient using Credentials pulled from `process.env` (`WATSONX_API_KEY`, `WATSONX_URL`). This ensures that if the code is ever audited or committed, no secrets are leaked.¹
-

6. Data Engineering: Multimodal Ingestion and Vector Search

The ability to process PDFs directly is a game-changer for this MVP. The "Multimodal" capability removes the need for fragile OCR (Optical Character Recognition) pipelines.

6.1 Handling PDF Inputs

The research confirms that models like `pixtral-12b` and `llama-3-2-11b-vision-instruct` support PDF inputs via Base64 encoding.¹

- **The Ingestion Pipeline:**
 1. **Multer Middleware:** The Node.js server uses the `multer` library to handle the multipart/form-data upload from the React client. It buffers the file in memory.
 2. **Base64 Conversion:** The file buffer is converted to a Base64 string.
 3. **Payload Construction:** The API request is constructed with the specific media type header:

```
JSON
{
  "role": "user",
  "content":
}
```
 4. **Context Management:** To avoid re-sending this massive string for every follow-up question (which would be slow and expensive), the system performs a "One-Shot Extraction." The PDF is processed once, and a detailed text summary/JSON extraction is generated. *Only this extracted text* is added to the session history for subsequent chat turns.

6.2 Moderation and Governance

The AI output must be safe for the enterprise. The API provides **Moderation Controls**.¹

- **HAP (Hate and Profanity):** The MVP enables the HAP filter with a low threshold to ensure professional output.
 - **PII (Personally Identifiable Information):** The research highlights a PII filter that can "mask" entities.¹
 - *Configuration:* { "pii": { "mask": { "remove_entity_value": true } } }.
 - *Application:* This is enabled for the *logging* of data to ensure that while the system processes the candidate's name for the user, it does not write sensitive personal data to the system logs, maintaining GDPR/CCPA compliance.
-

7. User Experience: The React-Based Interaction Layer

The "Real-Time Value" is ultimately perceived through the UI. The React application is designed to be responsive, intuitive, and transparent.

7.1 The "One Minute" Workflow

The UX is optimized for speed. The "One Minute" success criterion ¹ dictates the flow:

1. **0:00 - Upload:** The user drags the file onto the **UploadZone** component.
2. **0:05 - Processing:** A "Skeleton Loader" appears, indicating that the Document Reader Agent is active.
3. **0:15 - Rendering:** The **SummaryDashboard** component populates.
 - *Visual Hierarchy:* The most critical information (Readiness Score, Missing Items) is displayed at the top in red/green indicators.
4. **0:30 - Action:** The user reviews the "Recommended Actions" list (e.g., "Request Passport").
5. **0:45 - Drafting:** The user clicks "Generate Welcome Email." The **Communication Agent** streams the text into a rich-text editor.
6. **0:55 - Completion:** The user sends the email and clicks "Sync to SAP" (simulated in

MVP), completing the task.

7.2 Dynamic State Management

React's state management capabilities are leveraged to handle the asynchronous nature of the agents.

- **Optimistic UI:** When the user clicks "Sync to SAP," the UI immediately shows a "Syncing..." state, providing instant feedback while the Node.js backend performs the actual API calls.
- **Live Updates:** As the backend processes the document, it can push partial updates (using Server-Sent Events or WebSockets) to the client. For example, "Compliance Check Complete... Now Checking Org Structure..." This transparency builds trust in the AI's process.

7.3 The "Chat with Document" Feature

The interface includes a persistent "Assistant Panel."

- **Context Awareness:** Because the Node.js backend maintains the session history, the user can ask, "Does this contract mention a relocation bonus?" The assistant answers based on the specific document context, not general knowledge.
- **Rich Responses:** The research notes the ability to "Configure rich responses".¹ The React app is built to render not just text, but structured elements like buttons ("Draft Email") or links ("View Policy") returned by the Assistant, making the chat actionable.

8. Roadmap to Maturity: From MVP to Predictive Intelligence

The MVP described above solves the immediate pain of document review. However, the "Extended Version" of the use case ¹ points toward a future where the system becomes a strategic asset.

8.1 Phase 2: Dynamic Version Tracking

Onboarding is rarely linear. Contracts are revised; dates change.

- **Feature:** The system will track document versions. When a revised contract is uploaded, the **Document Reader Agent** will perform a "Diff" analysis, highlighting *only* what changed (e.g., "Start Date moved from Dec 1 to Dec 15").
- **Value:** This prevents the HRBP from having to re-read the entire document, saving massive amounts of rework time.

8.2 Phase 3: Global Compliance Engine

The "World Data" and "Visa" agents¹ enable the system to scale globally.

- **Feature:** The **Compliance Review Agent** will be upgraded with region-specific logic. If the "World Data" agent detects the location as "France," the system loads the "French Labor Law" rule set via RAG.
- **Value:** This ensures that a centralized HR team can onboard employees in multiple geographies without needing local legal experts for every single document review.

8.3 Phase 4: Predictive Analytics and Retention

This is the ultimate goal of the "Extended Scope".¹

- **Feature:** By aggregating data from the "Onboarding Summaries" over time, the system can identify patterns.
- **Insight:** "Candidates with a 'readiness score' below 80% on Day One have a 30% higher attrition rate in the first year."
- **Value:** This moves HR from reactive processing to proactive retention strategies, flagging "At-Risk" new hires before they even start.

9. Conclusion

The **AI-Powered HR Onboarding Document Assistant** represents a convergence of operational necessity and technological capability. By moving beyond the limitations of manual processing and passive chatbots, the solution utilizes **Watsonx Orchestrate** to create an Agentic workflow that is intelligent, autonomous, and integrated.

The proposed hybrid architecture—combining the interactive speed of **React**, the robust state management of **Node.js**, and the cognitive power of **Watsonx**—provides a stable foundation for this transformation. By leveraging the rich ecosystem of **SAP SuccessFactors agents**, the MVP delivers immediate, real-time value, turning isolated document insights into integrated enterprise actions. This is not just an efficiency tool; it is the first step toward a self-driving HR function, where technology handles the administration, liberating humans to focus on the art of talent management.

Works cited

1. Use Case Name_ AI-Powered HR Onboarding Document Assistant (MVP Build) (1).pdf